

Credit Risk Classification System

Technical Report for FinTech Solutions Inc.

Prepared by: Teerth Gangwar

Trimester 2 End-of-Term Project: Machine Learning Pipeline Development and Deployment

Dataset: German Credit Risk Dataset (1,000 applications)

1. Executive Summary

FinTech Solutions Inc. currently assesses loan applications manually, a process that takes three to five days per application and produces inconsistent decisions across loan officers. This project delivers an automated credit risk classification system that predicts whether a loan applicant is likely to default, using the German Credit dataset: 1,000 historical applications across 10 features.

Four classification models were built and compared: Logistic Regression, Decision Tree, Random Forest, and XGBoost. Random Forest and XGBoost were additionally tuned using 5-fold cross-validated grid search. All six resulting runs, four baseline models plus two tuned variants, were tracked with MLflow, including parameters, metrics, and confusion matrix artifacts.

XGBoost (tuned) comes out ahead on every metric measured: accuracy (0.875), recall (0.823), F1 (0.839), and AUC-ROC (0.953). At a decision threshold of 0.45, chosen deliberately over both the default 0.50 cutoff and a threshold that simply maximizes precision, it delivers 84.8% recall on defaults at 85.9% precision, comfortably clearing the business's 75% recall requirement. Random Forest tells a different story. Despite posting the highest precision of any model tested (0.902), both its baseline and tuned versions fall short of the 75% recall requirement (70.9% and 69.6% respectively), which rules it out for production regardless of its other strengths.

Recommendation: deploy XGBoost (tuned) at a 0.45 classification threshold, with the monitoring and retraining plan described in Section 6.

2. Data Analysis

2.1 Data Quality

The dataset contains 1,000 applications across 10 original features, with no duplicate rows. Two columns carry missing values as flagged in the project brief: Saving accounts (85 missing, 8.5%) and Checking account (95 missing, 9.5%). Both were treated as informative-missing rather than random noise, since missingness itself can correlate with risk, and were imputed with an explicit "unknown" category rather than a guessed value or row deletion.

The target variable is moderately imbalanced: 60.4% "good" risk versus 39.6% "bad" risk.

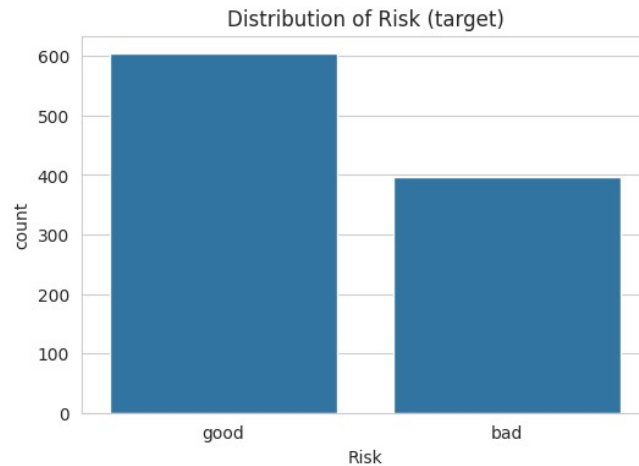


Figure 1. Target distribution: Risk (good vs. bad).

2.2 Exploratory Data Analysis: Key Insights

Distributions of Age, Credit amount, and Duration are all close to uniform rather than the right-skewed shape often seen in lending data, so no log transform was needed for these features.

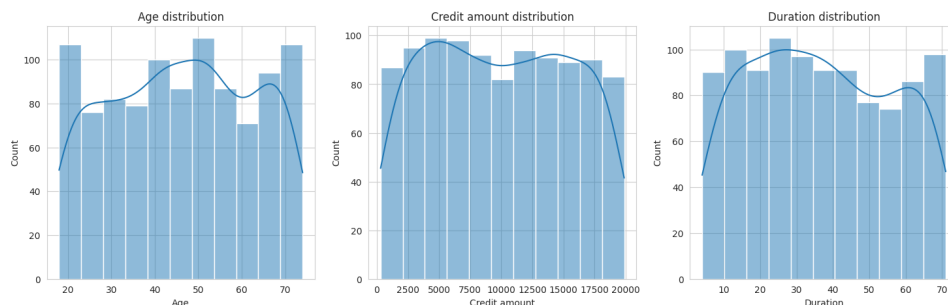


Figure 2. Distributions of Age, Credit amount, and Duration.

Boxplots by risk category show the clearest separation on Credit amount and Duration: bad-risk applicants skew toward larger, longer loans. Age shows little separation on its own.

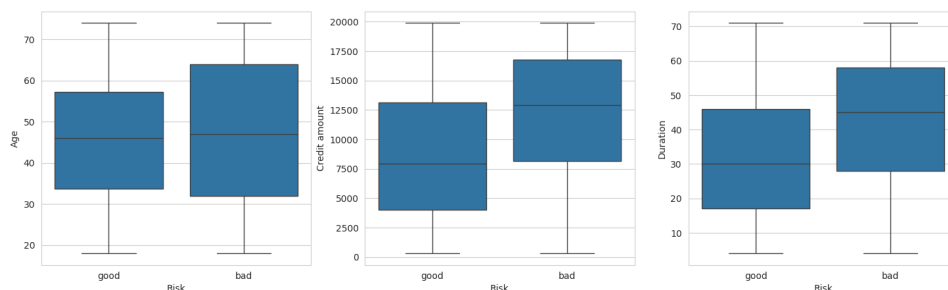


Figure 3. Age, Credit amount, and Duration by risk category.

The correlation heatmap among numeric features (Age, Job, Credit amount, Duration) shows negligible pairwise correlation across the board, with all values between -0.08 and 0.01. That is unusual for credit data, where Credit amount and Duration typically show a moderate positive relationship. It means the engineered ratio features in Section 2.3 combine largely independent signals rather than capturing a strong existing relationship.

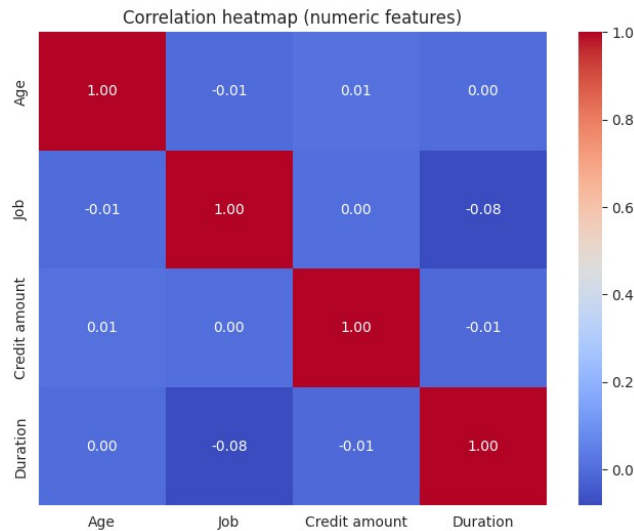


Figure 4. Correlation heatmap, numeric features.

Checking account status is the strongest categorical signal identified in the EDA. Applicants with "little" balance show roughly a 50% bad-risk rate, compared to about 27% for "moderate" and 25% for "rich", close to a twofold difference.



Figure 5. Checking account status vs. risk.

Loan purpose versus risk was examined to inform feature engineering. Only "business" showed a bad-risk share clearly above the roughly 40% baseline, at close to 50/50, while other purposes such as education and vacation/other sat near baseline. The engineered High_Risk_Purpose feature in Section 2.3 was narrowed to "business" only on this evidence, rather than including purposes on assumption alone.

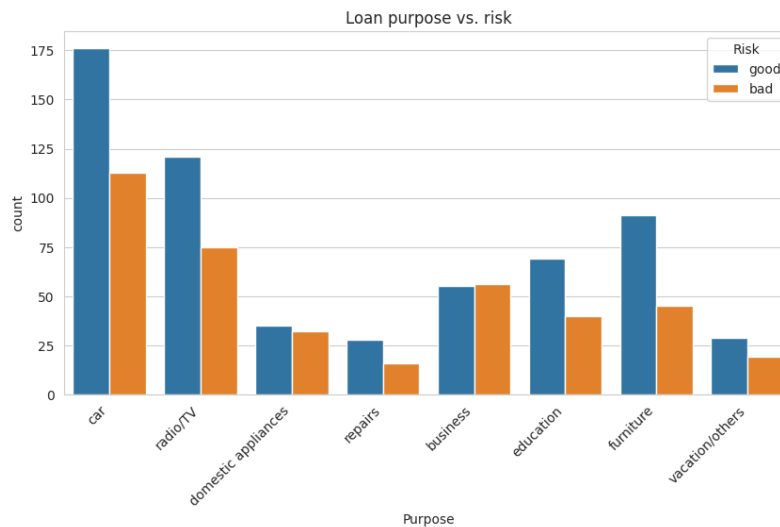


Figure 6. Loan purpose vs. risk.

2.3 Feature Engineering

- **Credit_to_Duration_Ratio**: credit amount divided by loan duration, which captures repayment burden rather than either raw figure alone.
- **Age_Group**: bucketed age (18-25, 26-35, 36-50, 50+), meant to capture non-linear age effects for linear models.
- **High_Risk_Purpose**: a binary flag for "business"-purpose loans specifically, based on the EDA evidence in Figure 6 rather than a broader assumed list.
- **Account_Stability**: a combined savings and checking account score (0 = none or missing, higher = more stable), which collapses two related categorical columns into one interpretable signal.

Account_Stability in particular is validated by the modeling results. It ranks in the top three most important features for all three tree-based models in Section 4.4, confirming that the engineering decision added real predictive value rather than redundant signal.

3. Methodology

3.1 Preprocessing

Missing categoricals were imputed as "unknown". All remaining categorical columns were one-hot encoded, and the target was encoded as 0 for good and 1 for bad, so that recall throughout this report refers specifically to recall on the "bad" (default) class, the figure the business's 75% requirement targets. The data was split 80/20 (800 train, 200 test) with stratification on Risk to preserve class balance. StandardScaler was fit on the training set only and then applied to both splits, avoiding leakage from the test set.

3.2 Model Selection Rationale

Four models were built to span a range of interpretability and complexity, as required by the brief:

- Logistic Regression: the baseline model, fully interpretable through its coefficients and the fastest to retrain and audit.
- Decision Tree: a single interpretable tree, useful as a readability benchmark against the ensembles.
- Random Forest: an ensemble of trees, generally more robust than a single tree, with moderate interpretability through feature importance.
- XGBoost: a gradient-boosted ensemble, typically the strongest performer on tabular data of this kind, with interpretability supported through feature importance rather than direct coefficients.

3.3 Hyperparameter Tuning Strategy

Random Forest and XGBoost were tuned using GridSearchCV with 5-fold cross-validation, scored on F1. This balances precision and recall during the search, rather than optimizing recall alone, which could otherwise push a model toward flagging nearly every applicant as high risk.

- Random Forest grid: n_estimators [100, 200], max_depth [10, 20, None], min_samples_split [2, 5], min_samples_leaf [1, 2].
- XGBoost grid: learning_rate [0.01, 0.1], max_depth [3, 5, 7], n_estimators [100, 200].

Logistic Regression and Decision Tree were kept as untuned baselines. This is consistent with Logistic Regression's role as the project's explicit baseline model, and it gives a clean pre-tuning reference point for the two ensembles.

3.4 Experiment Tracking

All 6 model runs, 4 baseline plus 2 tuned, were logged to MLflow under a single experiment (credit_risk_classification). Each run was named separately and logged with its parameters, the 5 evaluation metrics, and a confusion matrix image artifact, giving a complete and auditable record of every model variant trained during this project.

4. Results

4.1 Model Comparison

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Logistic Regression	0.835	0.811	0.759	0.784	0.925
Decision Tree	0.825	0.775	0.785	0.780	0.818
Random Forest (tuned)	0.850	0.902	0.696	0.786	0.938
XGBoost (tuned)	0.875	0.855	0.823	0.839	0.953

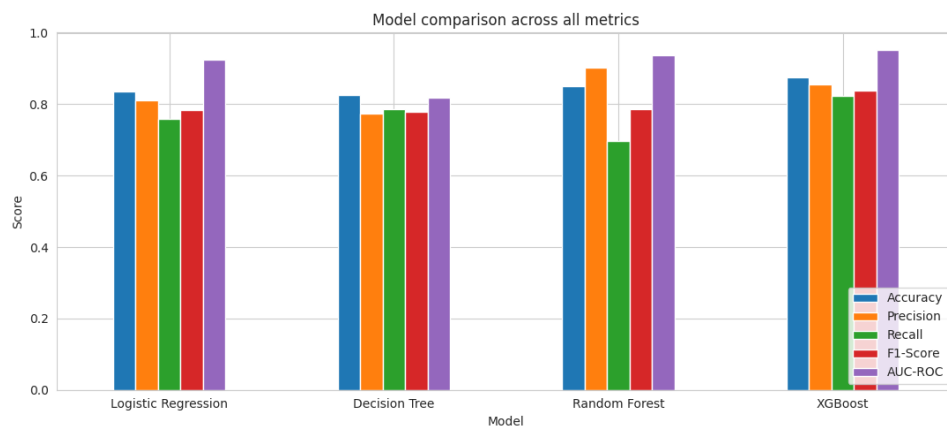


Figure 7. All 4 models across all 5 metrics.

4.2 Confusion Matrices

Confusion matrices reveal what the metrics mean in real terms: specifically, how many actual defaulters (bad risk) each model misses as false negatives.

Model	TN	FP	FN (missed defaults)	TP
Logistic Regression	107	14	19	60
Decision Tree	103	18	17	62
Random Forest (tuned)	115	6	24	55
XGBoost (tuned)	110	11	14	65

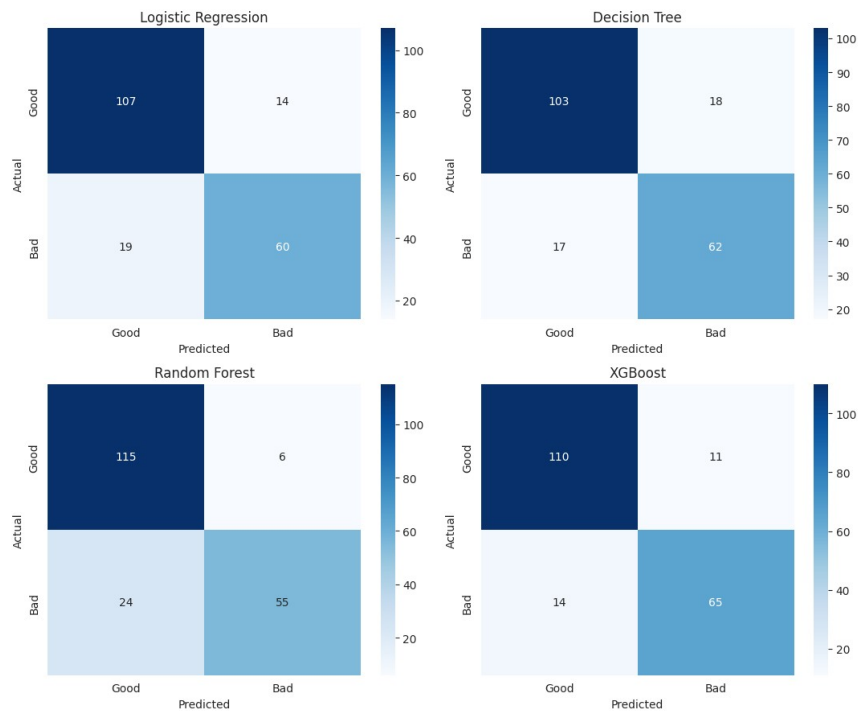


Figure 8. Confusion matrices for all 4 models (test set, $n=200$).

Random Forest has the fewest false positives (6) but the most missed defaulters (24 of 79 actual bad-risk cases). It is the most cautious of the four models about flagging an application as high risk, and given the cost asymmetry discussed in Section 5, that turns out to be the wrong kind of caution. XGBoost misses the fewest defaulters of any model, at 14 of 79.

4.3 ROC Curves

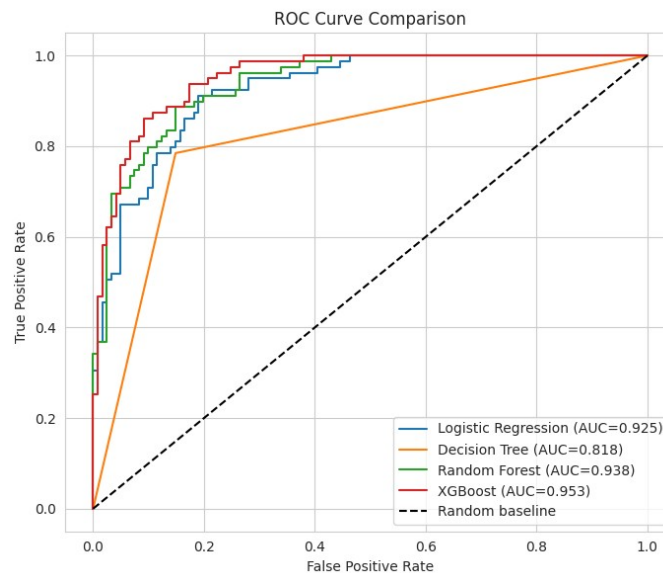


Figure 9. ROC curves, all 4 models.

XGBoost leads on AUC-ROC (0.953), followed by Random Forest (0.938) and Logistic Regression (0.925), with Decision Tree well behind at 0.818. The Decision Tree's visibly more angular curve reflects a structural property of single trees (a small number of distinct leaf probabilities) rather than a modeling error.

4.4 Feature Importance

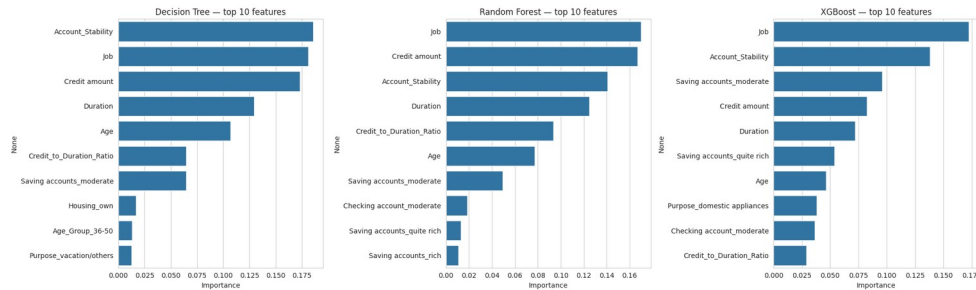


Figure 10. Top 10 feature importance: Decision Tree, Random Forest, XGBoost.

Job, an income and skill-level proxy that was not examined directly in the original EDA, turns out to be the single strongest predictor for both Random Forest and XGBoost, ahead of Credit amount and Duration. This does not contradict the EDA boxplots in Figure 3: those show univariate separation, while tree-based importance reflects each feature's contribution once the others are already accounted for. The engineered Account_Stability feature ranks in the top three for all three tree models, which is a concrete validation of that feature engineering choice.

4.5 Precision-Recall Curve and Threshold Analysis

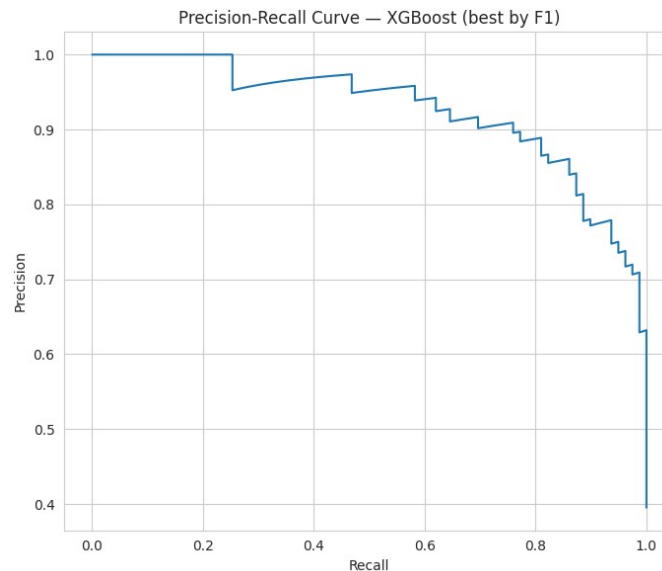


Figure 11. Precision-Recall curve for XGBoost (best model by F1).

Since the 75% recall requirement is as much a decision-threshold question as a model-choice question, classification thresholds from 0.10 to 0.90 were scanned on XGBoost's predicted probabilities:

Threshold	Precision	Recall	F1
0.10	0.690	0.987	0.812
0.20	0.771	0.937	0.846
0.30	0.778	0.886	0.828
0.40	0.840	0.861	0.850
0.45	0.859	0.848	0.854
0.50	0.855	0.823	0.839
0.55	0.884	0.772	0.824
0.60	0.908	0.747	0.819

Threshold	Precision	Recall	F1
0.70	0.915	0.684	0.783
0.80	0.958	0.582	0.724

The selection rule was to maximize F1 among thresholds that clear 75% recall, rather than simply maximizing precision among them. A precision-only rule would pick a threshold sitting right at the edge of the recall requirement (0.55, for 77.2% recall), leaving little safety margin against future model drift. Threshold 0.45, highlighted above, achieves the best F1 in the entire scan (0.854) while keeping a healthier recall margin of 84.8% above the 75% floor.

5. Business Recommendations

5.1 Which metric matters most, and why

Recall on the "bad" (default) class matters most. Accuracy is misleading given the roughly 60/40 class imbalance: a model that rarely predicts "bad" can still post high accuracy while missing most real defaulters, which is exactly the failure mode the business can least afford.

5.2 Cost asymmetry: false positives versus false negatives

The two error types are not comparable in cost. A false negative, approving an applicant who later defaults, risks the full loan principal plus lost interest. A false positive, rejecting an applicant who would have repaid, only costs the foregone profit margin on that one loan, since the applicant can seek credit elsewhere. This asymmetry is why the brief sets a recall floor rather than an accuracy or precision target, and why recall, not precision, is the metric that should not be compromised below 75%.

5.3 Recommended model and threshold

Recommendation: XGBoost (tuned), decision threshold 0.45. This model leads on every metric measured: accuracy, recall, F1, and AUC-ROC. At threshold 0.45 it delivers 84.8% recall, a healthy margin above the 75% floor, at 85.9% precision, the best F1 balance available in the threshold scan.

Random Forest, despite having the highest precision of any model (0.902 tuned), is disqualified: both its baseline and tuned versions fail the hard 75% recall requirement outright, at 70.9% and 69.6% respectively. A model that cannot meet the business's stated risk tolerance is not a production candidate, regardless of its other strengths.

Justification against the brief's four required criteria:

- **Performance:** XGBoost is the only model with clear, comfortable headroom above the 75% recall requirement at a deliberately chosen threshold.
- **Interpretability for regulatory compliance:** XGBoost is not as directly interpretable as Logistic Regression's coefficients, but feature importance (Figure 10) gives a defensible, auditable explanation of what drives its predictions.
- **Inference speed:** all four models predict a single application in milliseconds at this dataset's scale, so none is a bottleneck against the sub-one-second requirement.
- **Maintenance complexity:** XGBoost requires more care around versioning and drift monitoring than Logistic Regression, but the recall improvement justifies that added operational cost given the cost asymmetry in Section 5.2.

6. Deployment Considerations

6.1 Infrastructure

Containerize the model, with FastAPI serving the pickled XGBoost model through Gunicorn or Uvicorn workers. Host it on a managed platform: AWS Lambda for low or spiky traffic, or a small Kubernetes or ECS service for steadier traffic. Either option comfortably meets the sub-one-second inference requirement at this model's scale.

6.2 API Design

- POST /predict accepts a JSON payload of applicant features (Age, Sex, Job, Housing, account status, Credit amount, Duration, Purpose).
- Response: { "risk_probability": 0.83, "prediction": "bad", "threshold_used": 0.45 }
- Input validation rejects out-of-range values, such as negative age or duration, and unrecognized categorical values, and the expected schema is versioned so upstream changes do not silently break scoring.

6.3 Performance Monitoring

- Track API latency (p50/p95) against the sub-one-second requirement.
- Track realized recall and precision once loan outcomes are known, at loan term completion, not just at deployment time.
- Alert if realized recall drifts meaningfully below the 75% target over a rolling window.

6.4 Data Drift Detection

- Compare incoming applicant feature distributions against the training distribution on a regular cadence, for example weekly, using a tool such as Evidently AI or a per-feature KS-test.
- Watch particularly for shifts in Job, Account_Stability, Credit amount, and Duration, the features identified as most predictive in Section 4.4.

6.5 Retraining Strategy

- Scheduled retraining, for example quarterly, on newly labeled data from completed loans with known outcomes.
- Triggered retraining if drift monitoring crosses a defined threshold, rather than waiting for the next scheduled cycle.

6.6 A/B Testing Approach

- Shadow mode first: route a percentage of applications through the new model without acting on its predictions, and compare against actual outcomes.
- Ramp gradually, from shadow to partial live traffic to full rollout, keeping the ability to roll back quickly if realized recall or business metrics regress.

6.7 Potential Challenges

- Regulatory scrutiny of an ensemble model's explainability may call for supplementing feature importance with instance-level explanations, such as SHAP, for individual adjudication decisions.
- The near-zero feature correlations observed in the EDA (Figure 4) may not hold as the applicant population shifts over time, which could change which engineered features remain useful.
- The chosen threshold of 0.45 was optimized against the current test set. It should be revisited whenever the model is retrained, rather than assumed to remain optimal indefinitely.

7. Conclusion and Future Work

This project built a complete machine learning pipeline for credit risk classification, tracked and reproducible from raw data through to a production-ready model recommendation. XGBoost (tuned), deployed at a 0.45 decision threshold, meets FinTech Solutions Inc.'s 75% recall requirement with a healthy margin while remaining explainable and fast enough for real-time use.

Key learnings:

- Hyperparameter tuning does not uniformly improve every metric. Random Forest's tuning traded recall for precision, a real and reportable trade-off rather than an error.
- Feature importance surfaced a predictor, Job, that univariate EDA alone did not flag as important. That gap shows the value of running both analyses rather than relying on EDA alone.
- Threshold selection matters as much as model selection. The naive rule of maximizing precision subject to the recall floor would have picked a threshold with a thin safety margin against future drift.

Suggestions for future work:

- Add a dedicated Job-vs-Risk EDA visualization, since Job emerged as the top predictor during modeling despite not being examined directly during EDA.
- Explore SHAP values for instance-level explanations, which would strengthen the regulatory-compliance case for deploying an ensemble model.
- Test threshold stability across simulated population shifts before committing to 0.45 as a long-term production setting.
- Extend feature engineering with the two candidates identified but not yet implemented: a No_Financial_Account flag and a Job-adjusted loan burden ratio.

8. Appendix

Google Colab Notebook Links

Insert your shared "Anyone with the link can view" links here before submission:

- Notebook 1, EDA and Preprocessing: [link]
- Notebook 2, Model Development and Hyperparameter Tuning: [link]
- Notebook 3, Model Evaluation and Comparison: [link]

Google Drive Folder

Insert your "Anyone with the link can view" Drive folder link here.